

Simple Registration & Login API using .NET 8 + SQL Server + BCrypt

Introduction

This document explains how to build a simple registration and login Web API using .NET 8, SQL Server (Code First), and BCrypt password hashing.

Key Concepts:

- Single Entity (User)
- DTOs for input
- Password Hashing using BCrypt
- Entity Framework Core Code First

Architecture Overview

Flow:

1. Register

- Accept user input
- Hash password
- Save to DB

2. Login

- Validate email
- Verify password using BCrypt

NuGet Packages

Install:

- Microsoft.EntityFrameworkCore.SqlServer
- Microsoft.EntityFrameworkCore.Tools
- BCrypt.Net-Next

Model: User.cs

```
public class User
{
    public int Id { get; set; }
    public string Name { get; set; }
    public string Email { get; set; }
```

```
    public string PasswordHash { get; set; }  
}
```

DTOs

RegisterDto:

- Name
- Email
- Password

LoginDto:

- Email
- Password

DbContext

```
public class AppDbContext : DbContext  
{  
    public DbSet<User> Users { get; set; }  
}
```

Controller Logic

Register:

- Check email exists
- Hash password
- Save

Login:

- Validate email
- Verify password using BCrypt

Program.cs Setup

Configure services:

- AddControllers
- AddDbContext
- Swagger

Database Setup

Run:

Add-Migration InitialCreate

Update-Database

Testing

Use Swagger:

- POST /register
- POST /login

Important Note

Passwords are stored as hashed values using BCrypt. Never store plain text passwords.

CODE:

📁 SimpleAuthAPI/ Model/ User.cs

```
using System.ComponentModel.DataAnnotations;

namespace SimpleAuthAPI.Models
{
    public class User
    {
        [Key]
        public int Id { get; set; }

        [Required]
        public string Name { get; set; } = string.Empty;

        [Required]
        [EmailAddress]
        public string Email { get; set; } = string.Empty;

        [Required]
        public string PasswordHash { get; set; } = string.Empty;
    }
}
```

📁 SimpleAuthAPI/ DTOs/ RegisterDto.cs

```
using System.ComponentModel.DataAnnotations;

namespace SimpleAuthAPI.DTOs
{
    public class RegisterDto
    {
        [Required]
        public string Name { get; set; } = string.Empty;

        [Required]
        [EmailAddress]
        public string Email { get; set; } = string.Empty;

        [Required]
        [MinLength(6)]
        public string Password { get; set; } = string.Empty;
    }
}
```

📁 SimpleAuthAPI/ DTOs/ LoginDto.cs

```
using System.ComponentModel.DataAnnotations;

namespace SimpleAuthAPI.DTOs
{
    public class LoginDto
    {
        [Required]
        [EmailAddress]
        public string Email { get; set; } = string.Empty;

        [Required]
```

```

        public string Password { get; set; } = string.Empty;
    }
}

```

📁 SimpleAuthAPI/ Data/ AppDbContext.cs

```

using Microsoft.EntityFrameworkCore;
using SimpleAuthAPI.Models;

namespace SimpleAuthAPI.Data
{
    public class AppDbContext : DbContext
    {
        public AppDbContext(DbContextOptions<AppDbContext> options) : base(options)
        {
        }

        public DbSet<User> Users { get; set; }
    }
}

```

📁 SimpleAuthAPI/ Controllers/ AuthController.cs

```

using Microsoft.AspNetCore.Mvc;
using Shared.DTOs;
using Shared.Models;
using UserService.Services;

namespace UserService.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class AuthController : ControllerBase
    {
        private readonly IAuthService _authService;

        public AuthController(IAuthService authService)
        {
            _authService = authService;
        }

        [HttpPost("login")]
        public async Task<ActionResult<ApiResponse<AuthResponse>>> Login([FromBody] LoginRequest request)
        {
            var result = await _authService.Login(request);

            if (result == null)
            {
                return Unauthorized(ApiResponse<AuthResponse>.ErrorResponse("Invalid username or password"));
            }

            return Ok(ApiResponse<AuthResponse>.SuccessResponse(result, "Login successful"));
        }

        [HttpPost("register")]
        public async Task<ActionResult<ApiResponse<AuthResponse>>> Register([FromBody] RegisterRequest request)
        {
            var result = await _authService.Register(request);

            if (result == null)
            {
                return BadRequest(ApiResponse<AuthResponse>.ErrorResponse("Username or email already exists"));
            }
        }
    }
}

```

```

        }
        return Ok(ApiResponse<AuthResponse>.SuccessResponse(result, "Registration
successful"));
    }
}

```

SimpleAuthAPI/ appsettings.json

```

{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "ConnectionStrings": {
    "DefaultConnection":
"Server=(localdb)\\mssqllocaldb;Database=SimpleAuthDB;Trusted_Connection=True;MultipleActiveResultSets=true"
  },
  "AllowedHosts": "*"
}

```

SimpleAuthAPI/ Program.cs

```

using Microsoft.EntityFrameworkCore;
using SimpleAuthAPI.Data;

var builder = WebApplication.CreateBuilder(args);

// Add services to the container.
builder.Services.AddControllers();

// Configure DbContext with SQL Server
builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));

// Add Swagger/OpenAPI
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

var app = builder.Build();

// Configure the HTTP request pipeline.
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();
app.UseAuthorization();
app.MapControllers();

app.Run();

```

OUTPUT:

 **Swagger**
OpenAPI

Select a definition **SimpleAuthAPI v1**

SimpleAuthAPI 1.0 OAS 3.0

<https://localhost:7080/swagger/v1/swagger.json>

Auth

POST /api/Auth/register

POST /api/Auth/login

Schemas

LoginDto

RegisterDto

POST /api/Auth/register

Parameters

No parameters

Request body

application/json

```
{
  "name": "Daniel",
  "email": "dani@sample.com",
  "password": "dani123"
}
```

Execute

Clear

Responses

Curl

```
curl -X 'POST' \
  https://localhost:7080/api/Auth/register \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "name": "Daniel",
    "email": "dani@sample.com",
    "password": "dani123"
  }'
```

Request URL

https://localhost:7080/api/Auth/register

Server response

Code

Details

200

Response body

```
{
  "message": "User registered successfully",
  "userId": 1
}
```

Response headers

```
content-type: application/json; charset=utf-8
date: Mon, 11 Apr 2024 12:40:54 GMT
server: Kestrel
```

Responses

Code	Description	Links
200	OK	No links

POST /api/Auth/login

Parameters

No parameters

Request body

application/json

```
{
  "email": "dani@example.com",
  "password": "dani123"
}
```

Execute

Clear

Responses

Curl

```
curl -X 'POST' \
  https://localhost:7000/api/auth/login \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "email": "dani@example.com",
    "password": "dani123"
  }'
```

Request URL

https://localhost:7000/api/auth/login

Server response

Code

Details

200

Response body

```
{
  "message": "Login successful",
  "userId": 1,
  "name": "Daniel",
  "email": "dani@example.com"
}
```

Response headers

```
content-type: application/json; charset=utf-8
date: Mon, 13 Apr 2026 12:41:53 GMT
server: Kestrel
```

Responses

Code	Description	Links
200	OK	No links

Schemas

LoginDto

RegisterDto

	Id	Name	Email	PasswordHash
	1	Hari Shankar	hari@example.c...	\$2a\$11\$6byo0...
	2	Daniel	dani@example....	\$2a\$11\$LwuqS6...